# G6 Pokedex Database

**Team 6**:

Lucas Attias

Spencer Mantis

Justin Forkert

Syracuse University

IST-659 Data Admin Concept & Db Mgmt

Professor Gregory Michael Zink

June 18, 2024

**Table of Contents**

Overview Narrative.....3

Data Analysis of the Facts.....4

Conceptual Data Model.....5

Logical Data Model.....5

Data Questions and Answers.....6

Application Screens.....11

Team Log.....12

Reflective Conclusion.....13

# Overview Narrative

## Introduction

This database aims to support the game of catching Pokémon, connecting players to Pokémon, types, base stats and generation. Users can create an account to track the different Pokémon that they have caught. Users are also able to query the database without an account and use the G6 Pokedex as a resource while they play.

## Purpose of the Database

The purpose of this database is to be a reference point for Pokémon gamers. The database will link Pokémon and the user. It will allow users to view the attributes of different Pokémon, types, and generations. Players can develop battle strategies based on their queries and target specific Pokémon that users desire to catch.

## Target Audience

Gamers who want to track which Pokémon they have caught and look up the stats for different Pokémon. For the competitive gamer looking to get an edge, but also the casual fan who wants to inquire more about different Pokémon.

## Database Structure Overview

### High Level Business Requirements

- Provide a user-friendly platform to connect users to their Pokémon
- Provide a way to easily search for Pokémon by different attributes

### Functional Requirements

- User Registration and Authentication
- Searching for Pokémon
    - For specific Pokémon
    - For Pokémon of specific types
    - Ability to query by generation and/or base stat with an advanced search
- Pokémon inserts
- Saved Pokémon watchlist

# Data Analysis of the Facts

Below is a data analysis of the facts listing the entities, attributes, and relationships in the data model.

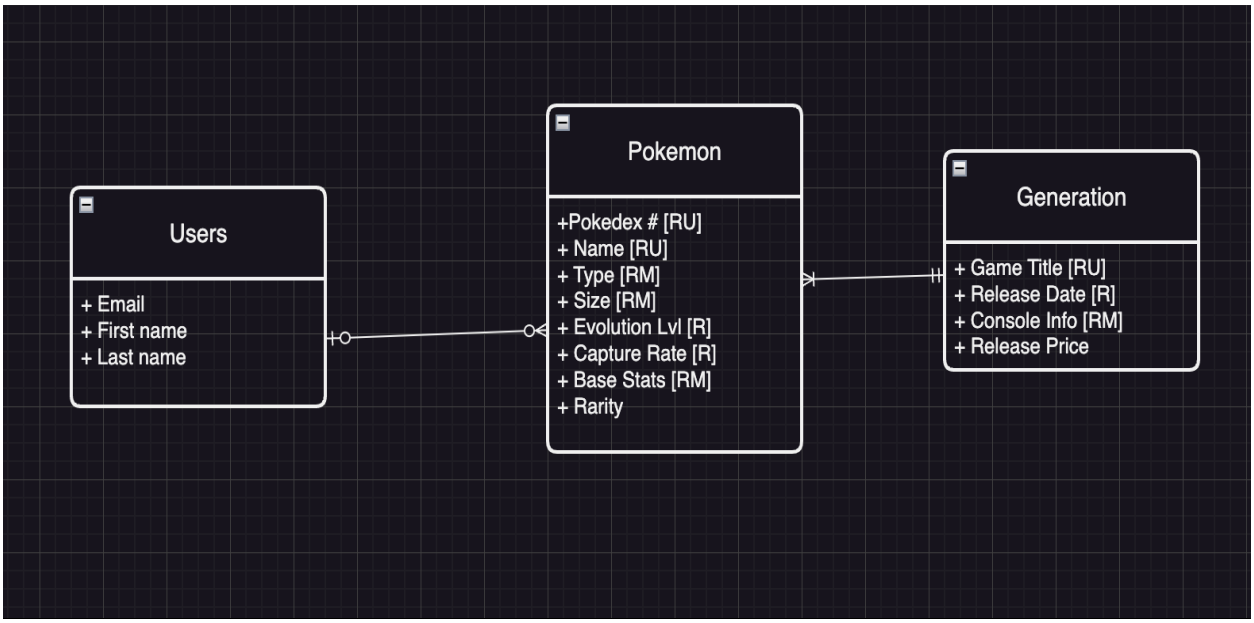
Entities & Attributes:

| Entity | Attribute | Props | Description |
|---|---|---|---|
| Pokemon | Pokedex # | RU | Unique identification # for each Pokemon |
| | Name | RU | Name of Pokemon |
| | Type | RM | All types of a Pokemon |
| | Size | RM | Height and weight of Pokemon |
| | Evolution Level | R | Level Pokemon evolves at |
| | Capture Rate | R | Base rate to catch Pokemon with no damage done |
| | Base Stats | RM | HP, speed, physical/special attack, physical/special defense of Pokemon |
| | Rarity | | How rare a Pokemon is to find |
| | Caught | R | Has the user caught It? |
| | Watchlist | | Add Pokemon to watch list |
| | | | |
| User | First Name | R | |
| | Last Name | R | |
| | Email | RU | |
| | | | |
| Generations | Game Title | RU | Game title generation was introduced in |
| | Game Release Date | R | Date of generation release |
| | Release Price | | Price of game at release |

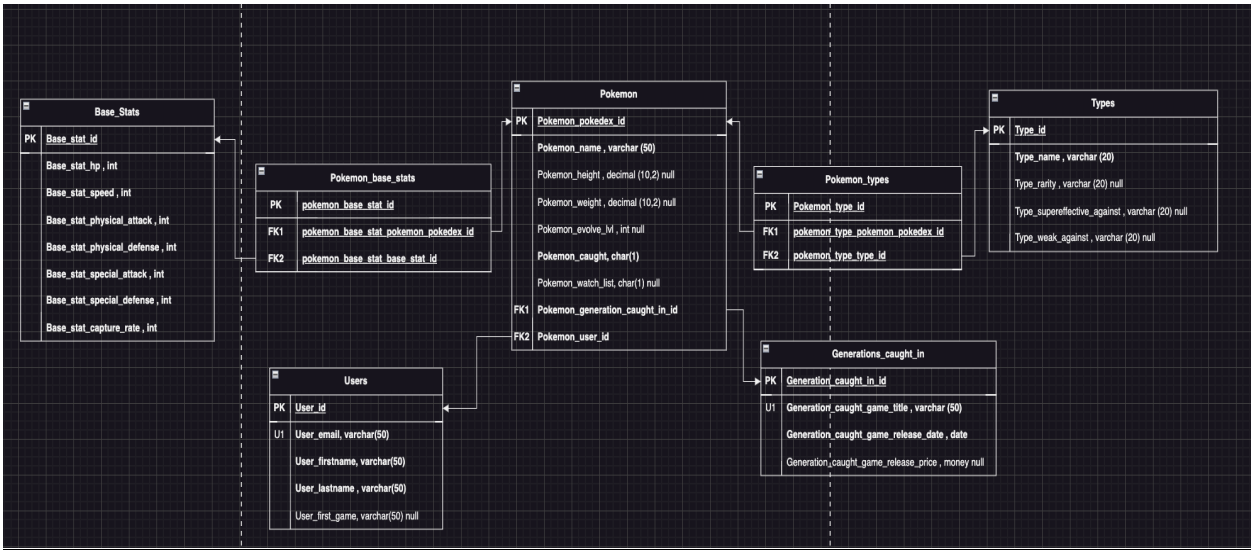
Relationships:

| Relationship | Entity | Rules | Min | Max | Entity 2 |
|---|---|---|---|---|---|
| Pokemon introduced in generation | Pokemon | Is introduced in | 1 | 1 | Generation |
| | Generation | Introduces | 1 | M | Pokemon |
| | | | | | |
| User and Pokemon | User | Has | 0 | M | Pokemon |
| | Pokemon | Has | 0 | 1 | User |

## Conceptual Data Model




## Logical Data Model



## Data Questions and Answers

1. Display Pokémon Pokedex ID, Pokemon name, Pokemon type for Pokémon that are super effective against the type 'Ground'

```sql
SELECT p.pokemon_pokedex_id, p.pokemon_name, t.type_name, t.type_supereffective_against
FROM pokemon p
JOIN pokemon_types pt ON p.pokemon_pokedex_id = pt.pokemon_type_pokemon_pokedex_id
JOIN types t ON t.type_id = pt.pokemon_type_type_id
JOIN pokemon_base_stats pbs ON pbs.pokemon_base_stat_pokemon_pokedex_id = p.pokemon_pokedex_id
WHERE t.type_supereffective_against LIKE '%Ground%'
```

Results | Messages

| | pokemon_pokedex_id | pokemon_name | type_name | type_supereffective_against |
|---|---|---|---|---|
| 1 | 149 | Dragonite | Flying | Grass/Ground/Bug/Ghost |
| 2 | 334 | Altaria | Flying | Grass/Ground/Bug/Ghost |
| 3 | 373 | Salamence | Flying | Grass/Ground/Bug/Ghost |
| 4 | 384 | Rayquaza | Flying | Grass/Ground/Bug/Ghost |

2. Display the fastest Pokémon in generation 2

```sql
/*Fastest pokemon in generation 2*/
SELECT TOP 1 p.pokemon_name, bs.base_stat_speed
FROM pokemon p
JOIN pokemon_base_stats pbs ON p.pokemon_pokedex_id = pbs.pokemon_base_stat_pokemon_pokedex_id
JOIN base_stats bs ON pbs.pokemon_base_stats_base_stat_id = bs.base_stat_id
WHERE p.pokemon_generation_caught_in_id = 2
ORDER BY bs.base_stat_speed DESC
GO
```

Results | Messages

| | pokemon_name | base_stat_speed |
|---|---|---|
| 1 | Kingdra | 85 |

3. Write a common table expression to show every Pokémon that is super effective against each type

```sql
/*cte for super effective against every type*/
WITH SuperEffectivePokemons AS (
    SELECT
        p.pokemon_name,
        CASE
            WHEN t.type_supereffective_against LIKE '%Ground%' THEN 'Ground'
            WHEN t.type_supereffective_against LIKE '%Electric%' THEN 'Electric'
            WHEN t.type_supereffective_against LIKE '%Bug%' THEN 'Bug'
            WHEN t.type_supereffective_against LIKE '%Dark%' THEN 'Dark'
            WHEN t.type_supereffective_against LIKE '%Dragon%' THEN 'Dragon'
            WHEN t.type_supereffective_against LIKE '%Fight%' THEN 'Fight'
            WHEN t.type_supereffective_against LIKE '%Fire%' THEN 'Fire'
            WHEN t.type_supereffective_against LIKE '%Flying%' THEN 'Flying'
            WHEN t.type_supereffective_against LIKE '%Ghost%' THEN 'Ghost'
            WHEN t.type_supereffective_against LIKE '%Grass%' THEN 'Grass'
            WHEN t.type_supereffective_against LIKE '%Ice%' THEN 'Ice'
            WHEN t.type_supereffective_against LIKE '%Normal%' THEN 'Normal'
            WHEN t.type_supereffective_against LIKE '%Poison%' THEN 'Poison'
            WHEN t.type_supereffective_against LIKE '%Psychic%' THEN 'Psychic'
            WHEN t.type_supereffective_against LIKE '%Rock%' THEN 'Rock'
            WHEN t.type_supereffective_against LIKE '%Steel%' THEN 'Steel'
            WHEN t.type_supereffective_against LIKE '%Water%' THEN 'Water'
            WHEN t.type_supereffective_against LIKE '%Fairy%' THEN 'Fairy'
            ELSE ''
        END AS super_effective_type
    FROM pokemon p
```

Results Messages

| | super_effective_against_ground | super_effective_against_electric | super_effective_against_bug | super_effective_against_dark | super_effective_against_dragon | super_effective_ |
|---|---|---|---|---|---|---|
| 1 | Altaria | Altaria | | | | |
| 2 | | Bagon | | | | |
| 3 | | Dragonair | | | | |
| 4 | Dragonite | Dragonite | | | | |
| 5 | | Dratini | | | | |
| 6 | | Flygon | | | | |
| 7 | | Kingdra | | | | |
| 8 | | Latias | | | | Latias |
| 9 | | Latios | | | | Latios |
| 10 | | | | | | |
| 11 | Rayquaza | Rayquaza | | | | |
| 12 | Salamence | Salamence | | | | |
| 13 | | Shelgon | | | | |
| 14 | | Vibrava | | | | |

4. Include a TVF for users who frequently filter Pokémon by a specific type

```sql
/* TVF for users who frequently filter Pokémon by a specific type*/
/*Drop Function*/
IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'dbo.GetPokemonByType') AND type in (N'FN', N'IF', N'TF', N'FS', N'FT'))
    DROP FUNCTION dbo.GetPokemonByType;
GO

/*Create function*/
CREATE FUNCTION dbo.GetPokemonByType(@type_id int)
RETURNS TABLE
AS
RETURN (
    SELECT p.pokemon_pokedex_id,
           p.pokemon_name,
           p.pokemon_generation_caught_in_id,
           pt.pokemon_type_id,
           t.type_name,
           t.type_rarity_rank,
           t.type_supereffective_against,
           t.type_weak_agaianst
    FROM pokemon p
    JOIN pokemon_types pt ON p.pokemon_pokedex_id = pt.pokemon_type_pokemon_pokedex_id
    JOIN types t ON pt.pokemon_type_type_id = t.type_id
    WHERE pt.pokemon_type_type_id = @type_id
);
GO

/*Test Function, show all ground pokemon*/
SELECT *
FROM dbo.GetPokemonByType(10);
```

Results Messages

| | pokemon_pokedex_id | pokemon_name | pokemon_generation_caught_in_id | pokemon_type_id | type_name | type_rarity_rank | type_supereffective_against | type_weak_agaianst |
|---|---|---|---|---|---|---|---|---|
| 1 | 329 | Vibrava | 3 | 7 | Ground | 7 | Electric/Poison/Rock | Water/Grass/Ice/ |
| 2 | 330 | Flygon | 3 | 9 | Ground | 7 | Electric/Poison/Rock | Water/Grass/Ice/ |

5. Include a TVF for users who frequently filter Pokémon by a specific generation

```sql
/* TVF for suers who frequently filter pokemon by generation*/
/* Drop Function*/
IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'dbo.GetPokemonByGeneration') AND type in (N'FN', N'IF', N'TF', N'FS', N'FT'))
    DROP FUNCTION dbo.GetPokemonByGeneration;
GO

/*Create function*/
CREATE FUNCTION dbo.GetPokemonByGeneration(@generation_id int)
RETURNS TABLE
AS
RETURN (
    SELECT p.pokemon_pokedex_id,
           p.pokemon_name,
           p.pokemon_height,
           p.pokemon_weight,
           p.pokemon_evolve_level,
           p.pokemon_rarity,
           p.pokemon_caught,
           p.pokemon_watch_list,
           p.pokemon_generation_caught_in_id,
           pt.pokemon_type_id,
           t.type_name,
           t.type_rarity_rank,
           t.type_supereffective_against,
           t.type_weak_agaianst
    FROM pokemon p
    JOIN pokemon_types pt ON p.pokemon_pokedex_id = pt.pokemon_type_pokemon_pokedex_id
    JOIN types t ON pt.pokemon_type_type_id = t.type_id
    WHERE @generation_id IS NULL OR p.pokemon_generation_caught_in_id = @generation_id
);
GO

/* Test function, Retrieve Pokémon from Generation 2*/
SELECT *
FROM dbo.GetPokemonByGeneration(2);
```

Results | Messages

| | pokemon_pokedex_id | pokemon_name | pokemon_height | pokemon_weight | pokemon_evolve_level | pokemon_rarity | pokemon_caught | pokemon_watch_list | pokemon_generation_ |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 230 | Kingdra | 1.80 | 152.00 | NULL | Rare | N | NULL | 2 |
| 2 | 230 | Kingdra | 1.80 | 152.00 | NULL | Rare | N | NULL | 2 |

6. Add a transaction with a custom message and error that displays when a user's entered email is a duplicate

```sql
/*Transaction to check integrity of user inserts, for success include a unique email*/
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO users (user_email, user_firstname, user_lastname)
    VALUES ('jmaio@syr.edu', 'Jonathon', 'Maio');
    DECLARE @user_id INT;
    SET @user_id = SCOPE_IDENTITY();
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK;
    throw;
END CATCH

/*Check user was inserted*/
select * from users

/*Transaction to check the email for a user isn't duplicate, failure*/
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO users (user_email, user_firstname, user_lastname)
    VALUES ('jmaio@syr.edu', 'Jonathon', 'Maio');
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 THROW 50001, 'Duplicate Email',1
        ROLLBACK;
    throw;
END CATCH
```

Messages

```
9:49:47 PM    Started executing query at Line 555
              (0 rows affected)
              Msg 50001, Level 16, State 1, Line 9
              Duplicate Email
              Total execution time: 00:00:00.011
```

7. Add a transaction with a custom message and error that displays when a user attempts to insert a duplicate game title

```
185
186 /* Insert into generations_caught_in table*/
187 BEGIN TRY
188     BEGIN TRANSACTION;
189 insert into generations_caught_in (
190     generation_caught_in_game_title,
191     generation_caught_in_game_release_date,
192     generation_caught_in_game_release_price)
193 values
194     ('Red/Blue', '1998-09-28', $29.95),
195     ('Gold/Silver', '2000-10-15', $29.95),
196     ('Ruby/Sapphire', '2002-11-21', $34.99),
197     ('Diamond/Pearl', '2006-09-28', $39.99),
198     ('Black/White', '2010-09-18', $34.99),
199     ('X/Y', '2013-10-12', $39.99),
200     ('Sun/Moon', '2016-11-18', $59.99),
201     ('Sword/Shield', '2019-11-15', $59.99),
202     ('Scarlet/Violet', '2022-11-18', $59.99);
203         COMMIT;
204 END TRY
205 BEGIN CATCH
206     IF @@TRANCOUNT > 0 THROW 50002, 'Duplicate Game Title',1
207         ROLLBACK;
208     throw;
209 END CATCH
210 GO
211
212 /* Insert into types table*/
```

Messages

```
8:25:05 PM    Started executing query at Line 187
              (0 rows affected)
              Msg 50002, Level 16, State 1, Line 20
              Duplicate Game Title
              Total execution time: 00:00:00.010
```

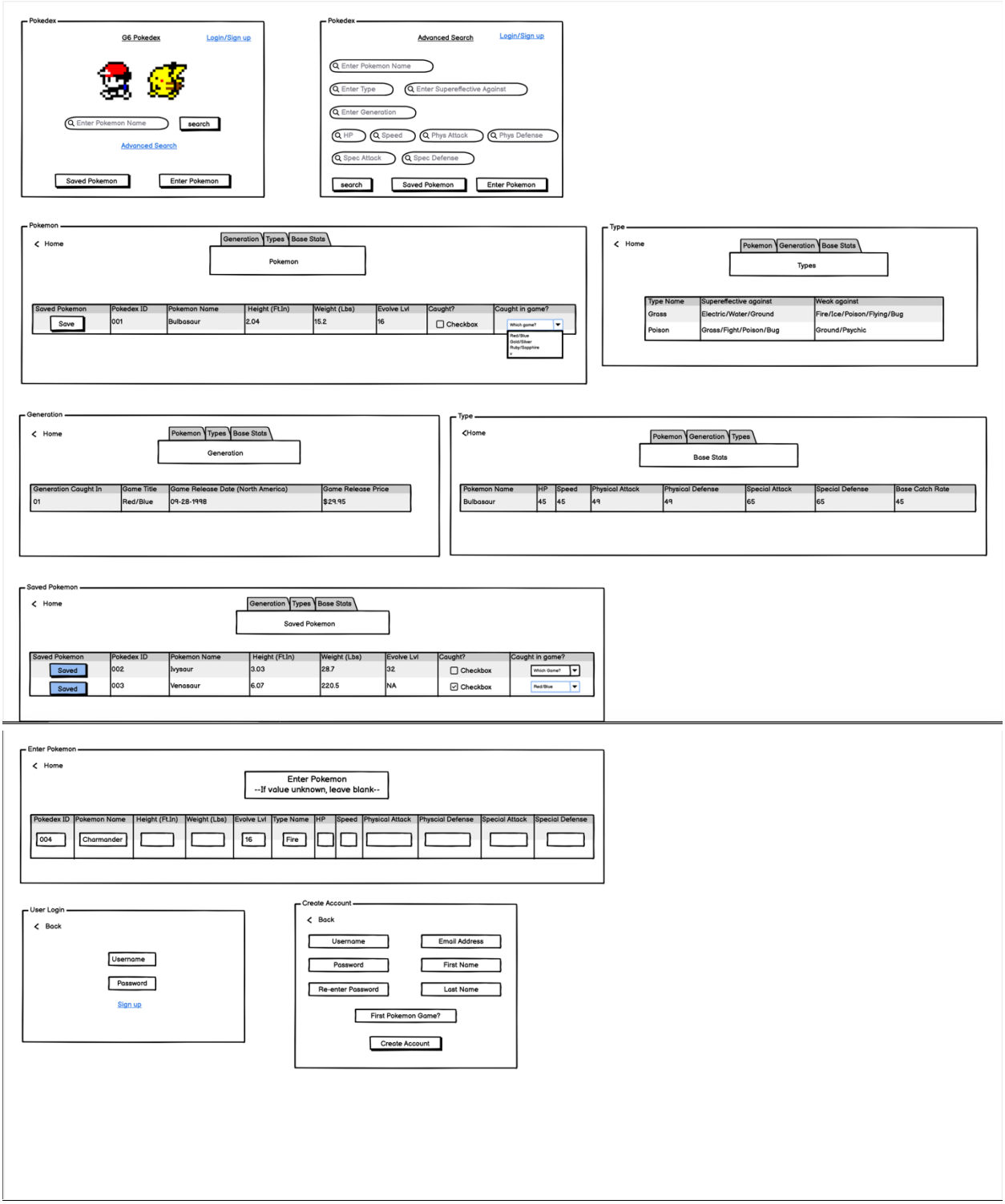
8. Insert the game title ‘Successful Transaction’ into the generations table successfully without any errors

```sql
BEGIN TRY
    BEGIN TRANSACTION;
insert into generations_caught_in (
    generation_caught_in_game_title,
    generation_caught_in_game_release_date,
    generation_caught_in_game_release_price)
values
    ('Successful Transaction', '1998-09-29', $29.98);
        COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 THROW 50002, 'Duplicate Game Title',1
        ROLLBACK;
    throw;
END CATCH
GO
select * from generations_caught_in
/* Insert into types table*/
```

Results Messages

| | generation_caught_in_id | generation_caught_in_game_title | generation_caught_in_game_release_date | generation_caught_in_game_release_price |
|---|---|---|---|---|
| 1 | 1 | Red/Blue | 1998-09-28 | 29.95 |
| 2 | 2 | Gold/Silver | 2000-10-15 | 29.95 |
| 3 | 3 | Ruby/Sapphire | 2002-11-21 | 34.99 |
| 4 | 4 | Diamond/Pearl | 2006-09-28 | 39.99 |
| 5 | 5 | Black/White | 2010-09-18 | 34.99 |
| 6 | 6 | X/Y | 2013-10-12 | 39.99 |
| 7 | 7 | Sun/Moon | 2016-11-18 | 59.99 |
| 8 | 8 | Sword/Shield | 2019-11-15 | 59.99 |
| 9 | 9 | Scarlet/Violet | 2022-11-18 | 59.99 |
| 10 | 11 | Successful Transaction | 1998-09-29 | 29.98 |

# Application Screens



**Team 6 Team Log**

4/26 – Formed Teams chat

5/4 – Decided on “Pokemon” topic for project and submitted proposal to professor

5/15 – Meeting  with team 6 members to discuss different models required for this project

5/21 – Meeting with team 6 members to discuss our created models and map out our next steps

5/29 – Completed Relationships Table, Conceptual Model, Logical Model using Draw.io and edited the models

6/1 – Discussed team member roles moving forward and remaining project requirements

6/5 – Completed presentation stage of project including application screens mockup using Balsamiq design tool

6/10 – Completed the up/down script and created the database G6_Pokedex

6/12 –  Inserted all values into the tables “types” and “generation_caught_in”. inserted all values into the tables “pokemon” and “base_stats”. Inserted values into all bridge tables and successfully tested the table joins

6/13 – Completed three queries in the G6_Pokedex.dbo, added values into the “user” table, checked the normalization of the data, and edited application screens mockup in Balsamiq to better align with the database created

6/14 – Completed three queries in the G6_Pokedex.dbo, and added transactions, views, and Indexes to the G6_Pokedex.dbo

6/15 – Completed executive slides deck and overview narrative, completed reflection, completed data questions and answers

## Reflective Conclusion

Team 6 has been working to create a Pokedex for gamers to utilize while playing through the Pokémon video games. With so many different Pokémon in existence having so many different attributes, it is currently very difficult to search for Pokémon by these attributes. It is very surprising that the most popular database currently for Pokémon, serebii.net, doesn't have this feature available to users already. Following the implementation of our database, Pokémon research becomes much more streamlined. Team 6 is very proud of the work put in to create this database and a lot of planning went into this project before the actual designing.

Once we created the entities and relationships the project mapped itself out very nicely. It was strategic for the team to start at a high level and gradually increase specificity. Small tweaks were made to the original plan after the work had started, but for the most part the design followed the plan. One issue we had as a group is we wanted to add so many features that we just simply did not have enough time to get them all done. This resulted in selectively choosing what features should be required for the database to do the job.

If more time was given, this database would exceed any resource that is currently available for researching Pokémon. If the database was completely populated, I have no doubt that it would be widely accepted and beloved by the Pokémon community. This community can be extremely critical and analytical, and team 6 created the G6 Pokedex database with the user in mind. The G6 Pokedex gives the user the edge they need to win more battles, catch more Pokémon, and be the very best that no one ever was.